

## Exercise 1: Control Structures

### DB creation and Data insertion:

```
SET SERVEROUTPUT ON SIZE 1000000;
```

```
CREATE TABLE customers (  
    customer_id NUMBER PRIMARY KEY,  
    customer_name VARCHAR2(100) NOT NULL,  
    date_of_birth DATE NOT NULL,  
    balance NUMBER(15,2) DEFAULT 0,  
    is_vip CHAR(1) DEFAULT 'N' CHECK (is_vip IN ('Y', 'N')),  
    email VARCHAR2(100),  
    phone VARCHAR2(15),  
    created_date DATE DEFAULT SYSDATE  
);
```

```
CREATE TABLE loans (  
    loan_id NUMBER PRIMARY KEY,  
    customer_id NUMBER NOT NULL,  
    loan_amount NUMBER(15,2) NOT NULL,  
    interest_rate NUMBER(5,2) NOT NULL,  
    loan_start_date DATE NOT NULL,  
    loan_due_date DATE NOT NULL,  
    loan_status VARCHAR2(20) DEFAULT 'ACTIVE',  
    CONSTRAINT fk_loan_customer FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

```
CREATE SEQUENCE seq_customer START WITH 1 INCREMENT BY 1;
```

```
CREATE SEQUENCE seq_loan START WITH 1 INCREMENT BY 1;
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)  
VALUES
```

```
(seq_customer.NEXTVAL, 'David Lee', DATE '1952-09-10', 16000.00, 'david.lee@email.com', '999-0001');
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)  
VALUES
```

```
(seq_customer.NEXTVAL, 'Olivia Martin', DATE '1965-01-25', 9500.00, 'olivia.martin@email.com',  
'999-0002');
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)  
VALUES
```

```
(seq_customer.NEXTVAL, 'James King', DATE '1958-07-14', 27000.00, 'james.king@email.com', '999-0003');
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)  
VALUES
```

```
(seq_customer.NEXTVAL, 'Ava Thompson', DATE '1989-06-11', 11000.00, 'ava.thompson@email.com',  
'999-0004');
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)  
VALUES
```

```
(seq_customer.NEXTVAL, 'William Scott', DATE '1946-10-05', 4800.00, 'william.scott@email.com',  
'999-0005');
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)  
VALUES
```

```
(seq_customer.NEXTVAL, 'Emma Harris', DATE '1993-12-01', 7000.00, 'emma.harris@email.com',  
'999-0006');
```

```
INSERT INTO loans (loan_id, customer_id, loan_amount, interest_rate, loan_start_date,  
loan_due_date) VALUES
```

```
(seq_loan.NEXTVAL, 1, 55000.00, 5.7, DATE '2024-01-10', SYSDATE + 18);
```

```
INSERT INTO loans (loan_id, customer_id, loan_amount, interest_rate, loan_start_date,  
loan_due_date) VALUES
```

```
(seq_loan.NEXTVAL, 2, 28000.00, 6.2, DATE '2024-02-25', SYSDATE + 12);
```

```
INSERT INTO loans (loan_id, customer_id, loan_amount, interest_rate, loan_start_date,
loan_due_date) VALUES
```

```
(seq_loan.NEXTVAL, 3, 72000.00, 5.0, DATE '2024-03-15', SYSDATE + 35);
```

```
INSERT INTO loans (loan_id, customer_id, loan_amount, interest_rate, loan_start_date,
loan_due_date) VALUES
```

```
(seq_loan.NEXTVAL, 4, 43000.00, 5.4, DATE '2024-04-20', SYSDATE + 22);
```

```
INSERT INTO loans (loan_id, customer_id, loan_amount, interest_rate, loan_start_date,
loan_due_date) VALUES
```

```
(seq_loan.NEXTVAL, 5, 18000.00, 6.7, DATE '2024-05-01', SYSDATE + 9);
```

```
COMMIT;
```

### **Scenario 1:**

```
DECLARE
```

```
total_checked NUMBER := 0;
```

```
discount_applied NUMBER := 0;
```

```
loan_rows_updated NUMBER := 0;
```

```
CURSOR cur_seniors IS
```

```
SELECT customer_id, customer_name,
```

```
TRUNC(MONTHS_BETWEEN(SYSDATE, date_of_birth)/12) AS age
```

```
FROM customers
```

```
WHERE EXISTS (
```

```
SELECT 1 FROM loans
```

```
WHERE loans.customer_id = customers.customer_id
```

```
);
```

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE('*** APPLYING DISCOUNT FOR SENIOR CUSTOMERS ***');
```

```
FOR rec IN cur_seniors LOOP
```

```
total_checked := total_checked + 1;
```

```
IF rec.age > 60 THEN
```

```
UPDATE loans
```

```
SET interest_rate = GREATEST(interest_rate - 1.0, 1.0)
```

```

WHERE customer_id = rec.customer_id AND loan_status = 'ACTIVE';

loan_rows_updated := SQL%ROWCOUNT;

discount_applied := discount_applied + loan_rows_updated;

DBMS_OUTPUT.PUT_LINE('Discounted: ' || rec.customer_name || ' (Age: ' || rec.age || ')');

ELSE

    DBMS_OUTPUT.PUT_LINE('Skipped: ' || rec.customer_name || ' (Age: ' || rec.age || ')');

END IF;

END LOOP;

DBMS_OUTPUT.PUT_LINE('Customers Processed: ' || total_checked);

DBMS_OUTPUT.PUT_LINE('Total Loans Updated: ' || discount_applied);

IF discount_applied > 0 THEN

    COMMIT;

END IF;

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE('Error in Scenario 1: ' || SQLERRM);

END;

/

```

**Output :**

```

Output:

*** APPLYING DISCOUNT FOR SENIOR CUSTOMERS ***
Discounted: David Lee (Age: 72)
Skipped: Olivia Martin (Age: 60)
Discounted: James King (Age: 66)
Skipped: Ava Thompson (Age: 36)
Discounted: William Scott (Age: 78)
Customers Processed: 5
Total Loans Updated: 3

```

**Scenario 2:**

```

DECLARE

vip_upgraded NUMBER := 0;

existing_vips NUMBER := 0;

not_qualified NUMBER := 0;

total_customers NUMBER := 0;

CURSOR vip_cursor IS

    SELECT customer_id, customer_name, balance, is_vip

    FROM customers;

BEGIN

DBMS_OUTPUT.PUT_LINE('*** PROCESSING VIP PROMOTIONS ***');

FOR cust IN vip_cursor LOOP

    total_customers := total_customers + 1;

    IF cust.balance > 10000 THEN

        IF cust.is_vip = 'N' THEN

            UPDATE customers SET is_vip = 'Y' WHERE customer_id = cust.customer_id;

            vip_upgraded := vip_upgraded + 1;

            DBMS_OUTPUT.PUT_LINE('Promoted to VIP: ' || cust.customer_name);

        ELSE

            existing_vips := existing_vips + 1;

        END IF;

    ELSE

        not_qualified := not_qualified + 1;

    END IF;

END LOOP;

DBMS_OUTPUT.PUT_LINE('Total Customers: ' || total_customers);

DBMS_OUTPUT.PUT_LINE('New VIPs: ' || vip_upgraded);

DBMS_OUTPUT.PUT_LINE('Already VIPs: ' || existing_vips);

DBMS_OUTPUT.PUT_LINE('Not Eligible: ' || not_qualified);

IF vip_upgraded > 0 THEN

    COMMIT;

END IF;

```

```

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE('Error in Scenario 2: ' || SQLERRM);

END;

/

```

**Output :**

```

Output:

*** PROCESSING VIP PROMOTIONS ***
Promoted to VIP: David Lee
Promoted to VIP: James King
Promoted to VIP: Ava Thompson
Total Customers: 6
New VIPs: 3
Already VIPs: 0
Not Eligible: 3

```

**Scenario 3:**

```

BEGIN

    DBMS_OUTPUT.PUT_LINE('*** UPCOMING LOAN DUE REMINDERS (NEXT 30 DAYS) ***');

    FOR due_rec IN (

        SELECT l.loan_id, l.loan_due_date, l.loan_amount,
               c.customer_name, c.email, c.phone

        FROM loans l

        JOIN customers c ON l.customer_id = c.customer_id

        WHERE l.loan_due_date BETWEEN SYSDATE AND (SYSDATE + 30)

        AND l.loan_status = 'ACTIVE'

        ORDER BY l.loan_due_date

    ) LOOP

        DBMS_OUTPUT.PUT_LINE('Reminder: ' || due_rec.customer_name ||

                               ' | Loan ID: ' || due_rec.loan_id ||

                               ' | Due: ' || TO_CHAR(due_rec.loan_due_date, 'DD-Mon-YYYY') ||

```

```

        ' | Amount: ₹' || TO_CHAR(due_rec.loan_amount, '999,999.00'));

DBMS_OUTPUT.PUT_LINE('-> Contact: ' || due_rec.email || ' / ' || due_rec.phone);

DBMS_OUTPUT.PUT_LINE('-----');

END LOOP;

EXCEPTION

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Error in Scenario 3: ' || SQLERRM);

END;

/

```

### Output :

```

Output:

*** UPCOMING LOAN DUE REMINDERS (NEXT 30 DAYS) ***
Reminder: William Scott | Loan ID: 5 | Due: 08-Jul-2025 | Amount: ??? 18,000.00
-> Contact: william.scott@email.com / 999-0005
-----
Reminder: Olivia Martin | Loan ID: 2 | Due: 11-Jul-2025 | Amount: ??? 28,000.00
-> Contact: olivia.martin@email.com / 999-0002
-----
Reminder: David Lee | Loan ID: 1 | Due: 17-Jul-2025 | Amount: ??? 55,000.00
-> Contact: david.lee@email.com / 999-0001
-----
Reminder: Ava Thompson | Loan ID: 4 | Due: 21-Jul-2025 | Amount: ??? 43,000.00
-> Contact: ava.thompson@email.com / 999-0004
-----

```

### Exercise 3: Stored Procedures

```
SET SERVEROUTPUT ON SIZE 1000000;
```

```

CREATE TABLE customers (
    customer_id NUMBER PRIMARY KEY,
    customer_name VARCHAR2(100) NOT NULL,
    date_of_birth DATE NOT NULL,
    balance NUMBER(15,2) DEFAULT 0,
    is_vip CHAR(1) DEFAULT 'N' CHECK (is_vip IN ('Y', 'N')),
    email VARCHAR2(100),
    phone VARCHAR2(15),

```

```
    created_date DATE DEFAULT SYSDATE  
);
```

```
CREATE TABLE loans (  
    loan_id NUMBER PRIMARY KEY,  
    customer_id NUMBER NOT NULL,  
    loan_amount NUMBER(15,2) NOT NULL,  
    interest_rate NUMBER(5,2) NOT NULL,  
    loan_start_date DATE NOT NULL,  
    loan_due_date DATE NOT NULL,  
    loan_status VARCHAR2(20) DEFAULT 'ACTIVE',  
    CONSTRAINT fk_customer_loan FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

```
CREATE SEQUENCE customer_seq START WITH 1 INCREMENT BY 1;
```

```
CREATE SEQUENCE loan_seq START WITH 1 INCREMENT BY 1;
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)  
VALUES
```

```
(customer_seq.NEXTVAL, 'Ankit Verma', DATE '1949-06-10', 18000.00, 'ankit.verma@email.com',  
'987-1111');
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)  
VALUES
```

```
(customer_seq.NEXTVAL, 'Nisha Patel', DATE '1962-09-05', 9200.00, 'nisha.patel@email.com', '987-  
1112');
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)  
VALUES
```

```
(customer_seq.NEXTVAL, 'Suresh Iyer', DATE '1957-02-15', 27000.00, 'suresh.iyer@email.com', '987-  
1113');
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)  
VALUES
```

```
(customer_seq.NEXTVAL, 'Rekha Nair', DATE '1987-01-25', 14500.00, 'rekha.nair@email.com', '987-  
1114');
```



```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)
VALUES
```

```
(customer_seq.NEXTVAL, 'Arvind Rao', DATE '1947-08-12', 4700.00, 'arvind.rao@email.com', '987-1115');
```

```
INSERT INTO customers (customer_id, customer_name, date_of_birth, balance, email, phone)
VALUES
```

```
(customer_seq.NEXTVAL, 'Deepika Sinha', DATE '1994-10-30', 8900.00, 'deepika.sinha@email.com', '987-1116');
```

```
INSERT INTO loans (loan_id, customer_id, loan_amount, interest_rate, loan_start_date,
loan_due_date) VALUES
```

```
(loan_seq.NEXTVAL, 1, 42000.00, 5.3, DATE '2024-01-12', SYSDATE + 22);
```

```
INSERT INTO loans (loan_id, customer_id, loan_amount, interest_rate, loan_start_date,
loan_due_date) VALUES
```

```
(loan_seq.NEXTVAL, 2, 25000.00, 6.1, DATE '2024-03-18', SYSDATE + 18);
```

```
INSERT INTO loans (loan_id, customer_id, loan_amount, interest_rate, loan_start_date,
loan_due_date) VALUES
```

```
(loan_seq.NEXTVAL, 3, 69000.00, 5.0, DATE '2024-02-28', SYSDATE + 40);
```

```
INSERT INTO loans (loan_id, customer_id, loan_amount, interest_rate, loan_start_date,
loan_due_date) VALUES
```

```
(loan_seq.NEXTVAL, 4, 39000.00, 5.4, DATE '2024-05-01', SYSDATE + 26);
```

```
INSERT INTO loans (loan_id, customer_id, loan_amount, interest_rate, loan_start_date,
loan_due_date) VALUES
```

```
(loan_seq.NEXTVAL, 5, 22000.00, 6.3, DATE '2024-04-14', SYSDATE + 12);
```

```
CREATE TABLE employees (
```

```
    employee_id NUMBER PRIMARY KEY,
```

```
    employee_name VARCHAR2(100),
```

```
    department_id NUMBER,
```

```
    salary NUMBER(10,2)
```

```
);
```

```
CREATE TABLE accounts (
```

```
    account_id NUMBER PRIMARY KEY,
```

```
customer_id NUMBER,  
balance NUMBER(15,2)  
);
```

```
INSERT INTO employees VALUES (11, 'Meera Joshi', 10, 52000);  
INSERT INTO employees VALUES (12, 'Tarun Nair', 10, 49000);  
INSERT INTO employees VALUES (13, 'Sneha Desai', 20, 61000);
```

```
INSERT INTO accounts VALUES (2001, 1, 3500);  
INSERT INTO accounts VALUES (2002, 2, 1200);
```

```
COMMIT;
```

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest AS  
BEGIN  
    UPDATE customers  
    SET balance = balance + (balance * 0.01)  
    WHERE balance > 0;  
    DBMS_OUTPUT.PUT_LINE('Monthly interest of 1% applied to all savings accounts.');
```

```
    COMMIT;  
END;  
/
```

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (  
    p_dept_id IN NUMBER,  
    p_bonus_percent IN NUMBER  
) AS  
BEGIN  
    UPDATE employees  
    SET salary = salary + (salary * p_bonus_percent / 100)  
    WHERE department_id = p_dept_id;
```

```
    DBMS_OUTPUT.PUT_LINE('Bonus of ' || p_bonus_percent || '% applied to department ' ||  
p_dept_id);  
    COMMIT;  
END;  
/  

```

```
CREATE OR REPLACE PROCEDURE TransferFunds(  
    p_from_account IN NUMBER,  
    p_to_account IN NUMBER,  
    p_amount IN NUMBER  
) AS  
    v_balance NUMBER;  
BEGIN  
    SELECT balance INTO v_balance FROM accounts WHERE account_id = p_from_account FOR  
UPDATE;  
  
    IF v_balance < p_amount THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient funds in source account.');    END IF;  
  
    UPDATE accounts SET balance = balance - p_amount WHERE account_id = p_from_account;  
    UPDATE accounts SET balance = balance + p_amount WHERE account_id = p_to_account;  
  
    DBMS_OUTPUT.PUT_LINE('Transferred ' || p_amount || ' from account ' || p_from_account || ' to  
account ' || p_to_account);  
    COMMIT;  
END;  
/  

```